Informe Técnico — Sistema de Gestión y Distribución de Donaciones Post-Desastre

Universidad EAFIT · Estructuras de Datos y Algoritmos · Evaluación Práctica 3

Autores

Mariana Cardona Vidal
Juan Pablo Marín Rondón
Viliam Sofía Mesa Angarita

Universidad EAFIT — Estructuras de Datos y Algoritmos — Evaluación Práctica 3

Autores:

- Mariana Cardona Vidal
- Juan Pablo Marín Rondón
- Viliam Sofía Mesa Angarita

---

1. Resumen

Se implementó un sistema que modela el flujo completo de la ayuda humanitaria post-desastre: recepción y categorización de donaciones, búsqueda multinivel, priorización por triaje, cálculo de rutas de despacho e informes en tiempo real. El diseño es modular (capas `models` -> `structures` -> `core` -> `gui`) y las cuatro estructuras de datos se implementaron desde cero para demostrar dominio del comportamiento algorítmico.

---

2. Fase 1 — Modelado y fuentes de datos

Entidades (`models/entities.py`)

Entidad	Atributos destacados	Decisiones
`Donante`	id, nombre, tipo, país	`frozen=True` (inmutable: un donante no cambia tras registrarse)
`LoteAyuda`	categoría, cantidad, peso, volumen, **requiere_frío**, **fecha_caducidad**, estado	Validación de rangos y coherencia temporal
`CentroAcopio`	capacidad, cadena de frío	—
`Municipio`	**nivel_urgencia**, población, **necesidades** (dict categoría->cantidad)	—

Validaciones (criterio "tipado estricto" de la rúbrica)

- **Tipos**: enums `CategoriaAyuda`, `EstadoLote`, `NivelUrgencia` con normalización desde `str`/`int`.
- **Rangos**: cantidades, peso y volumen no negativos; población y necesidades no negativas.
- **Coherencia temporal**: la fecha de caducidad debe ser **posterior al evento sísmico** (`EVENTO_FECHA = 2026-01-15`). Excepción documentada: los fondos económicos no caducan.

Persistencia (`core/repository.py`)

- Lectura/escritura **JSON** y lectura **CSV**.
- Manejo robusto de I/O: archivo inexistente, JSON corrupto y errores del SO se capturan y reportan sin lanzar excepciones no controladas.
- **Tolerancia a datos sucios**: un registro nulo o mal estructurado se registra en `CargaResultado.errores` y se omite, sin abortar la carga completa. Esto es crítico en una emergencia donde los datos llegan incompletos.

Pruebas: `test_models.py`, `test_repository.py` (registros nulos, campos faltantes, cantidades

negativas, caducidad previa al evento, JSON corrupto, round-trip de serialización).

---

3. Fase 2 — Especificación funcional

Fachada `SistemaDonaciones` (`core/system.py`). Catálogo de operaciones:

1. **Registro y categorización instantánea** — `registrar_donacion()`: valida donante, indexa el lote en el hash y en el árbol AVL por caducidad.
2. **Búsqueda multinivel** - por **identificador** -> `buscar_lote()` — O(1) vía hash. - por **categoría** -> `buscar_por_categoria()`. - por **caducidad próxima** -> `buscar_proximos_a_caducar()` — consulta de rango sobre el AVL.
3. **Asignación automatizada por triaje** — `asignar_recursos()`: extrae municipios por urgencia (min-max heap) y, para cada necesidad, toma lotes con política **FEFO** (*First-Expired-First-Out*, recorriendo el AVL en orden) calculando la ruta de despacho con Dijkstra.
4. **Informes en tiempo real** — `balance_stock()`, `despachados_vs_remanentes()`, `alertas_desabastecimiento()`.

Regla de negocio verificada (exigida por la rúbrica): un lote asignado cambia su estado a `ASIGNADO` y deja de aparecer como disponible (`test_lote_asignado_no_queda_disponible`). La conservación de inventario tras transacciones encadenadas se valida en `test_balance_consistente_tras_transacciones`.

---

4. Fase 3 — Estructuras de datos y análisis de complejidad

Notación: n = nº de elementos; para el grafo V = vértices, E = aristas; k = nº de resultados de una consulta de rango.

4.1 Tabla Hash — `hash_map.py`

- **Técnica**: encadenamiento separado (cada bucket es una lista de pares).
- **Redimensionado** al superar factor de carga 0.75 se duplica la capacidad y se re-hashea; al bajar de 0.20 se reduce.

Operación	Promedio	Peor caso	Justificación
`set` / `get` / `delete`	**O(1)** amortizado	O(n)	Con buena dispersión, cada bucket tiene O(1) elementos. El peor caso (todas las claves colisionan) degrada a recorrer un bucket de tamaño n. El redimensionado es O(n) puntual pero amortizado O(1) por inserción.
Espacio	O(n)	O(n)	Arreglo de capacidad proporcional a n.

Elección frente a direccionamiento abierto: el encadenamiento simplifica el borrado y degrada de forma más predecible bajo factores de carga altos, deseable ante ráfagas masivas de donaciones.

4.2 Min-Max Heap — `min_max_heap.py`

Montículo sobre un único arreglo con niveles alternados (pares = *min*, impares = *max*). Permite las dos consultas simétricas del triaje con un solo almacenamiento.

Operación	Complejidad	Justificación
`peek_min`	**O(1)**	El mínimo está en la raíz (índice 0).
`peek_max`	**O(1)**	El máximo es la raíz o el mayor de sus dos hijos.
`push`	**O(logn)**	Un `bubble-up` recorre como mucho la altura |log2 n|.
`pop_min` / `pop_max`	**O(logn)**	`bubble-down` comparando hijos y nietos.
Espacio	O(n)	Arreglo compacto.

Cada elemento se almacena como `(prioridad, contador, payload)`; el contador monotónico ofrece un desempate determinista y evita comparar los payloads.

Elección frente a dos heaps separados: un min-max heap da ambos extremos con la mitad de memoria y sin sincronización entre estructuras.

4.3 ÁrbolAVL —`avl_tree.py`

BST auto-balanceado; tras cada inserción/borrado se restaura la invariante de balance (|altura*izq ? altura*der| <=1) con rotaciones.

Operación	Complejidad	Justificación
`insert` / `search` / `delete`	**O(logn)**	La altura AVL está acotada por ~1.44·log2(n+2).
`range_query(lo, hi)`	**O(logn + k)**	Desciende a la frontera del rango y recorre k resultados in-order.
`min_key` / `max_key`	**O(logn)**	Camino más a la izquierda / derecha.
Espacio	O(n)	Un nodo por clave distinta.

Soporta **claves duplicadas**agrupando valores por nodo (varios lotes pueden compartir fecha de caducidad). Se eligió AVL por su estricto control de altura, que favorece las consultas de rango frecuentes del caso FEFO.

Verificación empírica del balanceo: `test_balanceo_tras_inserciones_ordenadas` inserta 1000 claves monotónicas (peor caso de un BST ingenuo, que degeneraría en lista O(n)) y confirma que la altura se mantiene <=1.45·log2(n+2).

4.4 Grafo+ Dijkstra—`graph.py`

Lista de adyacencia construida sobre nuestro propio `HashMap`. Dijkstra con cola de prioridad binaria mínima hecha a mano (`_MinPQ`, sin `heapq`).

Operación	Complejidad	Justificación
`add_vertex` / `add_edge`	O(1) amortizado	Inserción en hash maps.
`dijkstra`	**O((V + E) log V)**	Cada arista provoca a lo sumo un `push`; cada `push`/`pop` cuesta O(log V).
Espacio	O(V + E)	Listas de adyacencia.

No se usa A: no disponemos de heurística geográfica fiable para los nodos sintéticos; Dijkstra garantiza el óptimo sin heurística. El diseño permite añadir A* reutilizando* `_MinPQ` si se incorporan coordenadas.

4.5 Benchmarking (estructuras elegidas vs. ingenuas)

`tests/test_benchmark.py` contrasta empíricamente (resultados típicos; varían según la máquina):

Escenario	Alternativa ingenua	Estructura elegida	Mejora
500 extracciones del máximo sobre 20 000 datos	lista + `max()`/`remove()` ~ O(n²) -> ~0.19 s	min-max heap O(log n) -> ~0.06 s	~3×
200 consultas de rango sobre 50 000 claves	filtro de lista O(n) -> ~0.32 s	AVL O(log n + k) -> ~0.01 s	~28×

---

5. Fase 4 — Interfaz gráfica (Tkinter)

`gui/app.py` — panel de control con pestañas diferenciadas:

- **Inventario** tabla dinámica (`ttk.Treeview`) de lotes.
- **Registrar donación** formulario validado; errores de rango se muestran con `messagebox` (retroalimentación visual clara ante datos fuera de rango).
- **Asignación / Triaje**: ejecuta la asignación **en un hilo de trabajo** y vuelca el resultado con `root.after`, evitando el congelamiento de la UI.
- **Alertas**: consola de balance de stock y déficits.
- **Rutas de despacho**: cálculo visual de la ruta óptima entre dos nodos.

---

6. Plan integral de pruebas

72 pruebas automatizadas con `pytest`:

Archivo	Cobertura
`test_hash_map.py`	set/get/delete, colisiones, redimensionado y contracción, iteración
`test_min_max_heap.py`	min/max, extracción mixta, fuzz 200 elementos, casos vacíos
`test_avl_tree.py`	balanceo con inserción ordenada, rango, duplicados, borrado masivo balanceado
`test_graph.py`	Dijkstra, reconstrucción de ruta, nodo inalcanzable, peso negativo, dirigido
`test_models.py`	validación de tipos, rangos, coherencia temporal, round-trip
`test_repository.py`	registros nulos/corruptos, I/O, persistencia
`test_system.py`	triaje, FEFO, estados de lote, alertas, conservación de inventario
`test_benchmark.py`	rendimiento vs. alternativas ingenuas

Incluye **casos de borde**: estructuras vacías, un solo elemento, claves duplicadas, nodos aislados, datos fuera de rango.

7. Anexo de Transparencia Técnica (uso de IA)

> Conforme al numeral 3 del enunciado.

a. Prompts relevantes empleados

1. *"Crear el proyecto de la Evaluación Práctica 3 en Python"* —con el PDF del enunciado adjunto, para derivar fases, entidades y rúbrica.
2. Confirmación de tres decisiones de diseño: **(i)** implementar las estructuras de datos a mano (sin `dict`/`heapq` internos), **(ii)** usar **Tkinter** para la GUI, **(iii)** entrega completa con tests e informe.

b. Justificación técnica de las decisiones arquitecturales

- **Estructuras desde cero**: la rúbrica premia el dominio del comportamiento algorítmico; delegar en `dict`/`heapq` ocultaría el análisis de complejidad.
- **Encadenamiento separado** en el hash: borrado simple y degradación predecible bajo carga alta.
- **Min-max heap** en lugar de dos heaps: un solo arreglo cubre las dos consultas del triaje (más urgente / menos urgente).
- **AVL** sobre Red-Black: control de altura más estricto, favorable a las consultas de rango frecuentes del flujo FEFO.
- **Dijkstra** sobre A*: sin coordenadas no hay heurística admisible fiable; Dijkstra garantiza el óptimo.
- **Capas desacopladas** (`models`/`structures`/`core`/`gui`): permiten probar la lógica sin la GUI y sustituir estructuras sin tocar el negocio.
- **Asignación en hilo + `root.after`**: patrón estándar de Tkinter para no bloquear el bucle de eventos.

c. Demostración de dominio del código

Cada módulo incluye docstrings con la justificación de diseño y la complejidad. El equipo puede sustentar línea por línea: rotaciones AVL, el ajuste de `_size` al eliminar un nodo con dos hijos (el sucesor sube, no desaparece —ver `_delete`), el desempate por contador en el heap, y la relajación de aristas en Dijkstra. Los tests de borde evidencian la comprensión de los casos límite.

> **Nota de corrección detectada mediante pruebas**: durante el desarrollo, la suite reveló dos defectos que el equipo corrigió comprendiendo su causa raíz: (1) un doble decremento de `_size` en el borrado AVL de nodos con dos hijos, y (2) una expectativa errónea sobre la estabilidad del heap entre prioridades iguales. Ambos están documentados y resueltos.